

CS 2089M 数据库系统

2022–2023 春夏学期期末回忆卷

整合自两份回忆版本（CC98 论坛）*

题型：选择题 $8 \times 3' = 24'$ ，判断题 $6 \times 2' = 12'$ ，SQL 16'，E-R 设计 16'，并发控制 8'，ARIES 12'，Buffer Tree 12'。共 100'。

一、选择题（每题 3 分，共 8 题）

1. 对于如下定义的表（[\[补\]](#) 题目原文为“类似去年第二题”，以下为回忆版）：

```
create table course (  
    course_id char(2) primary key,  
    credit int,  
    prereq_id char(2),  
    foreign key prereq_id references course(course_id)  
        on delete cascade  
        on update cascade  
);
```

初始数据（[\[补\]](#) 来自 PDF 回忆版）：

course_id	credit	prereq_id
11	2	NULL
22	4	NULL
33	3	NULL
44	3	22
55	4	22
66	3	33

依次执行以下 SQL 语句：

- (1) INSERT INTO course VALUES ('77', 4, '11');
- (2) DELETE FROM course WHERE course_id = '11';
- (3) UPDATE course SET course_id = '11', credit = 2 WHERE course_id = '33';
- (4) SELECT COUNT(*) FROM course WHERE credit > 2;

最后一条语句返回的结果是：

A. 1

*版本一：较完整的 PDF 回忆版本；版本二：选择题 + 判断题 + 大题框架回忆版本。两版为同一套试卷，此处合并整理。

- B. 2
- C. 3
- D. 4

解：逐步追踪（注意 CASCADE 约束）：

初始： 11(2,NULL), 22(4,NULL), 33(3,NULL), 44(3,22), 55(4,22), 66(3,33)。

(1) INSERT ('77',4,'11'): 新增 77(4,11)。

(2) DELETE course_id='11': 删除 11。CASCADE: 所有 prereq_id=11 的记录也被删除
→ 77 被删。

剩余: 22(4,NULL), 33(3,NULL), 44(3,22), 55(4,22), 66(3,33)。

(3) UPDATE course_id='11', credit=2 WHERE course_id='33': 33 改为 11(credit=2)。
CASCADE: 66 的 prereq_id 跟着从 33 变为 11。

剩余: 11(2,NULL)^{原 33}, 22(4,NULL), 44(3,22), 55(4,22), 66(3,11)。

(4) SELECT COUNT(*) WHERE credit > 2: credit>2 的记录: 22(4), 44(3), 55(4), 66(3),
共 4 条。11 的 credit=2 不满足。

答案： D. 4。注意 CASCADE 的连锁效应是关键。

2. 对于上题定义的表 course，有如下 SQL 查询：

```
SELECT c1.course_id, c1.credit, c2.course_id, c2.credit
FROM course AS c1, course AS c2
WHERE c1.prereq_id = c2.course_id
      AND c1.credit = 4
      AND c2.credit > 2;
```

下列关系代数表达式中与上面的 SQL 查询**不等价**的是：^{[补] 选项回忆不全，考点是：两表连接属性不同名，不能直接用自然连接}
原卷选项缺失，无法确定具体选哪个。考点归纳见答案。

答案： 核心考点: $c1.prereq_id = c2.course_id$ 是两个**不同名属性**的等值连接，因此：

- $\pi(\sigma(\rho_{c1}(course) \times \rho_{c2}(course)))$ 形式的表达式是正确的
- 直接写 $\rho_{c1}(course) \bowtie \rho_{c2}(course)$ （自然连接）**不等价**——因为 c1 和 c2 虽出自同一表，但经重命名后，自然连接会基于**所有同名属性**做 join，而题目连接条件仅涉及 $prereq_id = course_id$ 一对不同名属性
- 必须显式写 $\bowtie_{c1.prereq_id=c2.course_id}$ ，不可用自然连接替代

3. 关系模式 $R(A, B, C, D, E, F)$ 上有函数依赖集 $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow B, D \rightarrow E, D \rightarrow F\}$ 。下列选项中是该关系模式的候选键的是：

- A. AC
- B. AD

C. AE

D. AF

解： $A^+ = \{A, B, C\}$, $D^+ = \{D, E, F\}$, $AD^+ = \{A, B, C, D, E, F\} = R$ 。任何不含 D 的集合无法推出 E, F ；任何不含 A 的集合无法推出存在 A, B, C 依赖性但已有 B, C 自循环。验证 AD 是最小—— $\{A, D\}$ 缺一不可。

答案：B. AD。

4. 函数依赖集 $\{A \rightarrow B, A \rightarrow D, BC \rightarrow E, AC \rightarrow D\}$ 的正则覆盖是：

[补] 选项回忆不全，以下为推算

A. $\{A \rightarrow BDE, BC \rightarrow E\}$

B. $\{A \rightarrow BD, ABC \rightarrow E\}$

C. $\{A \rightarrow BD, C \rightarrow E\}$

D. $\{A \rightarrow BD, BC \rightarrow E\}$

解：求正则覆盖：

步骤 1 (RHS 已单属性)： $\{A \rightarrow B, A \rightarrow D, BC \rightarrow E, AC \rightarrow D\}$ 。

步骤 2 (去掉 LHS 多余属性)：

- $AC \rightarrow D$ ：计算 $A_F^+ = \{A, B, D\}$ 。 $D \in A_F^+$ ，故 C 在 $AC \rightarrow D$ 中是多余的，变为 $A \rightarrow D$ (已存在)。
- $BC \rightarrow E$ ： $B_F^+ = \{B\}$ (无 E)， $C_F^+ = \{C\}$ (无 E)，均不多余。

步骤 3 (去掉冗余 FD)：得 $\{A \rightarrow B, A \rightarrow D, BC \rightarrow E\}$ ，即 $\{A \rightarrow BD, BC \rightarrow E\}$ 。

答案：D。考场上查 $BC \rightarrow E$ 中 B 或 C 是否多余、 $C \rightarrow E$ 是否可独立 (不能)，可用排除法。

5. 关系 A 上有 20,000 条记录。对关系 A 的非键属性建立 B^+ 树索引。每个 B^+ 树节点大小为 2KB，每个键值大小为 32B，每个指针大小为 8B。若某次查询得到的结果包含在 5 个块中，则此次查询的开销约为：

[补] 版本二备注：“基于此建立 B^+ 树非码上的索引”

A. 4 seek + 8 transfer

B. 4 seek + 9 transfer

C. 5 seek + 8 transfer

D. 5 seek + 9 transfer

解： B^+ 树节点容量：

- 非叶节点： $(n-1) \times 32 + n \times 8 \leq 2048 \Rightarrow n \leq 52$ ，扇出 = 52
- 叶节点： $n \times (32 + 8) + 8 \leq 2048 \Rightarrow n \leq 51$ ，每叶 51 条

20,000/51 $\approx$ 393 叶节点, $\lceil 393/52 \rceil = 8$ 个内部节点, $\rightarrow 1$ 个根。树高 3 (根 + L1 + 叶)。

查询开销 ([补] 设根节点缓存在内存):

- 索引遍历: L1 内节点 seek 1 + transfer 1, 叶节点 seek 1 + transfer 1 $\rightarrow 2$ seek + 2 transfer
- 结果 5 个数据块 (聚簇假设): 1 seek (定位首个块) + 5 transfer
- 合计: $2 + 1 = 3$ seek, $2 + 5 = 7$ transfer。与选项均不完美匹配。

[补] 实际考试可能按另一种假设 (根不缓存或结果块在索引叶中即为数据块)。因原卷数据完整但答案取决于假设, 考场建议现场推。最接近的估计为

答案: 推荐按 fanout 52、高 3、根 cached、结果 5 块索引 + 数据合计 8 或 9 transfer 估算, 选 A 或 B 均可 (需视考试当场假设)。重点掌握计算过程。

6. 关系 r 上有 50,000 条记录, 每个块可容纳 r 上的 200 条记录; 关系 s 上有 20,000 条记录, 每个块可容纳 s 上的 50 条记录。在不考虑递归分块的情况下, 对 r 和 s 进行 hash join 的开销约为:

- A. 1950 transfer + 1350 seek
- B. 1950 transfer + 650 seek
- C. 3300 transfer + 1350 seek
- D. 3300 transfer + 650 seek

解: $b_r = 50,000/200 = 250$ 块, $b_s = 20,000/50 = 400$ 块。

Hash join (不分块):

- 分区 r (读 + 写): $2 \times 250 = 500$ transfer
- 分区 s (读 + 写): $2 \times 400 = 800$ transfer
- Join 阶段 (读分区结果): $250 + 400 = 650$ transfer
- 总 transfer: $500 + 800 + 650 = \mathbf{1950}$

Seek 次数取决于缓冲区大小。若每块一个 seek (最坏): $(250 + 400) \times 2 = 1300$ seek (写分区时每 block 一次 seek)。加少量 join 阶段 seek ≈ 1350 。

答案: C. 1950 transfer + 1350 seek。[补] 若假设较大的缓冲区则 seek 可降至 650 左右, 选 B。实际考场需根据平时习题的缓冲区假设判断。

7. 关系 r 上有 30,000 条记录, 每个块上可容纳 200 条记录。若在 r 的候选键上进行等值查找, 则返回结果的大小最接近:

- A. 1
- B. 20
- C. 1,500
- D. 30,000

解: 候选键保证唯一性——等值查找最多命中 1 条记录。

答案: A. 1。

8. 在下面的锁请求序列中:

T1	T2	T3
lock-S(A)	lock-X(A)	lock-X(B)
	lock-X(B)	lock-S(C)
		lock-X(C)

[补] 原卷注明锁序列记不清了，写了类似的

下列说法正确的是：

- A. 计划中没有死锁
- B. 计划中有死锁，T1 在等待 T2 放锁的同时 T2 也在等待 T1 放锁
- C. 计划中有死锁，T2 在等待 T3 放锁的同时 T3 也在等待 T2 放锁
- D. 计划中有死锁，T2 在等待 T1 放锁，T3 在等待 T2 放锁，T1 在等待 T3 放锁

解：分析 wait-for 图：

- T1 持有 S(A)。T2 请求 X(A) → T2 等待 T1。
- T3 请求 X(B)，B 当前未被锁 → T3 获得 X(B)。
- T2 请求 X(B)，T3 持有 X(B) → T2 等待 T3。
- T3 持有 S(C)，请求 X(C) → 锁升级冲突（T3 等待自身或需先释放 S(C)），不构成跨事务等待。

Wait-for 边：T2 → T1（在 A 上），T2 → T3（在 B 上）。无环 → 无死锁。

[补] 注意：原卷锁序列为近似回忆，实际排布可能有所不同。解题关键是画 wait-for graph 判断有无环。

答案：在此回忆序列下选 A（无死锁）。实际考场按当时给出的序列画 wait-for 图即可。

二、判断题（每题 2 分，共 6 题）

1. 任意关系模式均可分解为三范式（3NF），且分解是保持依赖的。

解：正确。任何关系模式都可以无损地分解为 3NF，且保持函数依赖。这是 3NF 的经典定理——与 BCNF 不同（BCNF 不一定能保持依赖）。3NF 在「消除冗余」和「保持依赖」之间做了折中。

答案：√（正确）

2. 在事务频繁更新数据的场景下，按列存储的效率比按行存储高。

解：错误。列存储的优势在读密集型分析查询（只需少数列时节省 I/O），对频繁更新、插入的场景反而效率低——修改一行需要更新分散在不同列存储块中的多个位置，I/O 开销远大于行存储。

答案：×（错误）

3. 建立辅助索引的时候，为了降低空间开销，可以对每一个数据块建立稀疏索引。

解：错误。稀疏索引要求数据按搜索键**顺序存储**，通常只适用于主索引（聚簇索引）。辅助索引（非聚簇索引）的搜索键顺序与数据物理顺序不同，必须对每条记录建立索引项（稠密索引），不能使用稀疏索引。

答案：×（错误）

4. 连接前尽可能早地进行投影操作可以降低投影操作的开销，所以投影操作应该尽可能早做。

解：正确。投影尽早做（在查询树中下推）可缩减中间结果中的列数和行数，降低后续操作（连接、排序等）的 I/O 和计算开销。这是查询优化中“尽早投影”的启发式规则。

答案：√（正确）

5. 按索引查找的时间复杂度总是比顺序查找低。[补] 版本二忘掉序号 5，此为 PDF 版题

解：错误。“总是”过于绝对。在某些情况下（如小表、高选择性查询返回大量记录、或索引聚簇因子极差），顺序扫描可能比索引查找更快——索引查找需要先遍历 B⁺ 树再回表读数据块，多次随机 I/O 累计可能超过一次全表顺序扫描。

答案：×（错误）

6. 不能通过 2PL 产生的调度，可能可以由树协议（Tree Protocol）产生。

解：正确。树协议允许在访问数据项的树状结构中按特定规则加解锁（如必须先锁父节点才能锁子节点），可生成一些 2PL 无法产生的可串行化调度。两者产生的是不同的可串行化调度子集，互不包含。

答案：√（正确）

三、SQL 语句 (16 分)

对于如下定义的关系模式：

```
user(userID, name, gender, age, group_name)
followship(userID, followerID)
```

写出可以进行如下查询的 SQL 语句：

- (1) 查询所有含有男性成员的群组 (group)
- (2) 查询 userID 为'1001' 的用户的追随者的信息
- (3) 查出女性成员最多的群组
- (4) 查询名称为'game' 的群组当中，追随者 (follower) 数量高于该群组中所有用户的追随者数量的平均值的用户。

解：

- (1) 含有男性成员的群组：

```
SELECT DISTINCT group_name
FROM user
WHERE gender = 'male';
```

- (2) 1001 号用户的追随者信息：

```
SELECT u.*
FROM user u
JOIN followship f ON u.userID = f.followerID
WHERE f.userID = '1001';
```

- (3) 女性成员最多的群组：

```
SELECT group_name
FROM user
WHERE gender = 'female'
GROUP BY group_name
ORDER BY COUNT(*) DESC
LIMIT 1;
```

[补] 若有多组并列第一，需用子查询：

```

SELECT group_name
FROM user
WHERE gender = 'female'
GROUP BY group_name
HAVING COUNT(*) = (
    SELECT MAX(cnt) FROM (
        SELECT COUNT(*) AS cnt FROM user
        WHERE gender = 'female' GROUP BY group_name
    ) t
);

```

(4) game 群组中追随者数高于组内均值的用户:

```

SELECT u.userID, u.name,
       COUNT(f.followerID) AS follower_count
FROM user u
JOIN followship f ON u.userID = f.userID
WHERE u.group_name = 'game'
GROUP BY u.userID
HAVING COUNT(f.followerID) > (
    SELECT AVG(follower_count)
    FROM (
        SELECT COUNT(f2.followerID) AS follower_count
        FROM user u2
        LEFT JOIN followship f2 ON u2.userID = f2.userID
        WHERE u2.group_name = 'game'
        GROUP BY u2.userID
    ) AS avg_sub
);

```


四、数据库设计（16 分）

请根据以下需求设计一个跟团旅行预订平台：

- 每个用户都有用户 ID、用户名、密码和联系方式
- 每个旅行社需提供名称、法人代表身份证号、企业银行账号、位置，平台给每个旅行社分配 ID
- 旅行社会出售旅行计划，旅行计划的内容包含标题、时长、描述
- 每个用户都可以预订旅行计划，订单信息包含用户信息、旅行计划信息、订单发起时间、旅行计划数量、总价格、支付状态
- 旅行结束后用户可以对旅行社进行评价

(1) 画出 E-R 图。

(2) 将 E-R 图转化为关系模式，指明主键和外键。

解：

(1) **E-R 图** ([补] 文字描述, 考场画图):

实体：

- **User**：属性 $\{userID, name, password, contact\}$ ，userID 为键
- **TravelAgency**：属性 $\{agencyID, name, legal_ID, bank_account, location\}$ ，agencyID 为键
- **TravelPlan**：属性 $\{planID, title, duration, description\}$ ，planID 为键

联系：

- **Sell**：TravelAgency $\xrightarrow{1}$ Sell $\xrightarrow{N}$ TravelPlan（一个旅行社卖多个旅行计划）
- **Order**：User $\xrightarrow{M}$ Order $\xrightarrow{N}$ TravelPlan，带属性 $\{order_time, quantity, total_price, payment_status\}$
- **Rate**：User $\xrightarrow{M}$ Rate $\xrightarrow{N}$ TravelAgency，带属性 $\{rating, comment, rate_time\}$ （旅行后评价旅行社）

(2) **关系模式**：

- User(userID, name, password, contact)
- TravelAgency(agencyID, name, legal_ID, bank_account, location)
- TravelPlan(planID, title, duration, description, agencyID) FK: agencyID $\rightarrow$ TravelAgency
- OrderInfo(orderID, userID, planID, order_time, quantity, total_price, payment_status) FK: userID $\rightarrow$ User, planID $\rightarrow$ TravelPlan
- Rate(userID, agencyID, rating, comment, rate_time) FK: userID $\rightarrow$ User, agencyID $\rightarrow$ TravelAgency

五、并发控制（8 分）

[补] 原卷注明序列为近似回忆，以下为回忆序列

对于如下的访问序列：

$r_1(A) \ w_2(A) \ w_2(B) \ w_3(A) \ r_1(A) \ w_3(B) \ r_4(C) \ r_4(B) \ w_5(C) \ w_5(B) \ r_3(C)$

其中 $ri(D)$ 表示事务 T_i 对数据 D 进行读操作， $wi(D)$ 表示写操作。

- (1) 根据访问序列画出各个事务的前驱图 (precedence graph)
- (2) 判断这一访问序列是否冲突可串行化并说明理由
- (3) 判断这一访问序列是否可以由两阶段锁协议 (2PL) 产生并说明理由

解：

- (1) **前驱图**：找出所有冲突操作对（不同事务访问同一数据且至少一个是写）。

- $r_1(A) \rightarrow w_2(A) : T_1 \rightarrow T_2$
- $w_2(A) \rightarrow w_3(A) : T_2 \rightarrow T_3$
- $w_2(A) \rightarrow r_1(A)$ （第二条）： $T_2 \rightarrow T_1$
- $w_3(A) \rightarrow r_1(A)$ （第二条）： $T_3 \rightarrow T_1$
- $w_2(B) \rightarrow w_3(B) : T_2 \rightarrow T_3$
- $w_2(B) \rightarrow r_4(B) : T_2 \rightarrow T_4$
- $w_3(B) \rightarrow r_4(B) : T_3 \rightarrow T_4$
- $r_4(C) \rightarrow w_5(C) : T_4 \rightarrow T_5$
- $r_4(B) \rightarrow w_5(B) : T_4 \rightarrow T_5$
- $w_5(B) \rightarrow$ （无关） $r_3(C)$ ； $w_5(C) \rightarrow r_3(C) : T_5 \rightarrow T_3$

化简边： $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ （环）， $T_1 \rightarrow T_2 \rightarrow T_1$ （直接环）。

[补] 完整前驱图： T_1 - T_2 - T_3 三者两两成环， T_4 、 T_5 为下游。

- (2) **不可串行化**。前驱图存在环（ $T_1 \rightarrow T_2 \rightarrow T_1$ 或 $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ ），不满足冲突可串行化的充要条件（前驱图无环）。
- (3) **不可由 2PL 产生**。2PL 产生的所有调度都是冲突可串行化的（前驱图一定无环）。既然该序列不可串行化，就必然不能由 2PL 产生。

六、ARIES 恢复算法 (12 分)

[补] 原卷注明日志为近似回忆

给出如下的 ARIES 日志：

LSN	TID	Type	PageID	Slot	Before	After	prevLSN
1	T1	upd	6601	1	A	B	0
2	T2	upd	2628	2	C	D	0
3	T1	upd	2628	1	X	Y	1
4	T3	upd	6601	2	E	F	0
5	T1	upd	6601	3	G	H	3
6	T2	CLR					← checkpoint
7	T1	upd	...	...	...	...	5
8	T2	upd	...	...	...	...	6
9	T2	commit					8
10	T3	upd	6601	2	M	N	4

[补] 具体 LSN、PageID、Slot、值均无法完整回忆，以上日志结构仅用于说明考点

- (1) 写出在分析过程 (Analysis Phase) 中被加入到脏页表 (Dirty Page Table) 的记录
- (2) 重做过程从哪一条日志开始?
- (3) 写出恢复过程中增加的日志 (要求包含事务号、修改页号、slot 号和修改的值)
- (4) 恢复过程结束后, 6601.1 和 2628.2 的值分别是多少?

解: [补] 以下解答基于 ARIES 标准流程, 因原卷具体日志数据缺失, 无法给出确切数值。

- (1) **脏页表:** Analysis 阶段从 checkpoint 开始向前扫描, 将 checkpoint 时活跃事务涉及的所有已修改页加入脏页表, 再加上 checkpoint 之后的所有 update log 涉及的页。此处 checkpoint 前涉及页: 6601, 2628; checkpoint 后 LSN 7-10 也涉及 6601, 均入表。
- (2) **Redo 起点:** 脏页表中所有页的最早修改 LSN。6601 最早修改为 LSN 1, 2628 最早修改为 LSN 2, 取最小 = **LSN 1**。
- (3) **新增 CLR 日志:** Undo 阶段对 loser (未 commit 的 T3) 的每条 update, 按 prevLSN 链反向 undo。对每条 undo 写一条 CLR (Compensation Log Record), 含 undo 后的 before 值。
[补] 具体值依赖缺失的日志数据。
- (4) **最终值:** Redo 阶段从 LSN 1 重放到末尾 (包括 abort 产生的 CLR), 所有已 commit 事务 (T1, T2) 的更新生效, T3 被 undo 撤销。6601.1 最终为 Redo 后值, 2628.2 同理。[补] 具体值依赖缺失的日志数据。

答案: 考点: Analysis 确定脏页表 +loser/winner 集合; Redo 从最早脏页 LSN 起; Undo 沿 prevLSN 链反向撤销 loser; CLR 格式为补偿日志。答案形式对应实际日志。

七、Buffer Tree (12 分)

Buffer Tree 是一种写优化的索引结构——在 B^+ 树的每个内节点处增加一块缓冲区。插入键值时，先写入根节点缓冲区。当某节点缓冲区溢出时：

- 若子节点为内节点：将缓冲区所有元素下放到子节点缓冲区中
- 若子节点为叶子：将缓冲区所有元素插入子节点，子节点溢出时正常分裂

本题讨论缓冲区大小为 2 且度 $n = 4$ 的 Buffer Tree。初始状态如题图。

- (1) 现对该树做范围查询（不小于 30 且不大于 50），共有多少节点被访问？
- (2) 向该树中依次插入三个键值：38、92、28。画出三次插入操作之后的 Buffer Tree。
- (3) 在第 (2) 问的插入操作中，每次插入分别有多少节点被修改（分裂新增节点视同修改；根节点始终缓存于内存中，不计入）？

解：

[补] 原卷附有初始树图。因图片无法重现，以下基于 B^+ 树 $n = 4$ （每节点最多 4 个指针，3 个键）、buffer size=2 的标准 Buffer Tree 行为进行

- (1) **范围查询 [30, 50]**：从根沿树下降到叶子，遍历叶子链表。设初始树高为 h （含根节点），需访问根 + $h - 2$ 个内部节点 + 在范围 [30, 50] 内的所有叶子节点。[补] 具体节点数取决于初始树图。

- (2) **插入 38, 92, 28**：

- 插入 38：38 写入根 buffer（buffer 未满，不溢出）
- 插入 92：92 写入根 buffer → 根 buffer 满（2 个元素），触发下放。92 被下放到对应子节点（假设右子树）buffer，沿途 buffer 可能下放至叶子。若叶子满则分裂。
- 插入 28：28 写入根 buffer，触发下放到左子树，可能继续下放或分裂。

最终形态：插入过程可能产生叶子分裂（因 $n = 4$ ，叶满 4 元素时分裂为两个 2-元素的节点），并有部分键值滞留于各级 buffer 中。

[补] 因原卷有图，实际画图需根据初始 B^+ 树的具体键值和结构。此处仅描述推演过程。

- (3) **每次插入修改的节点数（不计根）**：

- 插入 38：仅写根 buffer，不触及磁盘节点 → 修改 0 个节点
- 插入 92：根 buffer 满 → 下放，修改被下放经过的各层节点及叶子，若叶子分裂则新增节点亦计数 → [补] 取决于树结构
- 插入 28：同理，触发下放和可能的分裂 → [补] 取决于树结构

答案：考点：Buffer Tree 通过集中批量下放减少磁盘 I/O，insert 不计根。解题关键是追踪 buffer 溢出触发下放的链式反应，以及叶子分裂时的新节点分配。